Fused Causal Attention with NVIDIA cuTile — Technical Report

Evidence snapshot: 20 October 2026
Implementation: src/cutile_attention.py
Target: NVIDIA GeForce RTX 4060, native Windows x64, compute capability 8.9
Core validation: GitHub Actions run #39422402438, commit bf5a6233e195c2ced647b4e20b5b2d914da60e8
Profiler export: run #39414502490, artifact #12261445684

> **Status:** Core and selected optional GPU correctness gates passed on 20 October. Assignment closure remains partial: PyTorch/SDPA comparison is blocked by a plain `torch.bmm` failure in the CUDA 13.0 framework environment; cold/JIT timings are missing for already-compiled long specializations; a medium-workload `LOAD_LATENCY=10` timing candidate awaits matched Nsight Compute review; and the individual-contribution statement needs author confirmation. No PyTorch speedup or vendor-attention result is claimed.

1. Executive summary

This project implements causal scaled dot-product self-attention as a genuine NVIDIA cuTile Python fused GPU kernel. The kernel keeps the full sequence-by-sequence score and probability matrices out of global memory, updates an online-softmax state across K/V tiles, skips wholly future tiles, and writes FP16 outputs.

Native RTX 4060 validation #39422402438 passed the real cuTile smoke (4,096 outputs), all 48 required attention tests with `REQUIRE_CUTILE=1`, 3 selected GQA/MQA and packed-variable-length tests, all 29 legal case/tile accuracy configurations, independent CLI/NumPy verification, and three controlled benchmark sessions. This run also captured two current NCU reports. Dedicated profiler run #39414502490 preserved eight compressed reports and seven actual Nsight Compute GUI screenshots. There was no RTX 4060 `ERR_NVGPUCTRPERM`.

The latest three-session benchmark measures the long D=128 case at tile 64×64 as **0.267059 ms median / 0.269128 ms P90 / 16.49×**, and the longer batched D=128 case at tile 64×128 as **0.915558 ms / 0.921457 ms / 18.69×**, all versus the deliberately unfused FP32 CuPy baseline. Across-session ranges were 0.92% and 0.50%; maximum baseline bracket drift was 0.03% and 0.08%. These are not comparisons with FlashAttention or vendor-optimized SDPA.

2. Objective and kernel design

Causal self-attention is

$$ s_{ij}=\begin{cases}\frac{q_i\cdot k_j}{\sqrt{D}},&j\le i\\-\infty,&j>i\end{cases},\quad p_{ij}=\frac{e^{s_{ij}}}{\sum_{j'\le i}e^{s_{ij'}}},\quad o_i=\sum_{j\le i}p_{ij}v_j. $$

Q/K/V use FP16 storage and logical layout [B,H,N,D]. QK^T accumulations, running-softmax state, normalizers and P×V accumulator are FP32; probabilities are converted to FP16 operands for P×V; outputs are FP16. Supported dimensions are $D \in \{32,64,128\}$. Sequence lengths must divide the selected tile dimensions; partial edge tiles are outside the fixed assignment scope.

Online-softmax derivation

For a query row, maintain a running maximum (m), normalization sum (ell), and unnormalized weighted-value accumulator (u). Initially (m=-\infty), (ell=0), and (u=0). For the next masked score tile (S_t), calculate its maximum (m_t), then (m'=\max(m,m_t)). Rescale the prior state by (\alpha=e^{m-m'}) and let (P_t=e^{S_t-m'}), with causally masked entries set to zero. Update

$$ \ell'=\alpha\ell+\sum_j P_{t,j},\qquad u'=\alpha u+P_tV_t. $$

Set (m\leftarrow m'), (\ell\leftarrow\ell'), (u\leftarrow u'); after the last K/V tile, output (o=u/\ell). Rescaling the old numerator and denominator to the new maximum lets each tile join the same normalized softmax result without storing an N×N matrix. The running maximum stabilizes the exponentials.

A program instance owns one query tile for a batch/head and query-block index. It loads Q[Tq,D], iterates K[Tk,D] and V[Tk,D], creates a temporary score tile [Tq,Tk], applies the causal mask, updates per-row softmax state, and stores O[Tq,D]. The loop bound avoids K/V tiles wholly beyond the final causal position in the query tile.

3. Correctness and numerical accuracy

The required workflow explicitly sets REQUIRE_CUTILE=1, so a missing cuTile import is a hard failure, not a silent GPU skip. A full-matrix FP32 NumPy oracle independently checks the output under the fixed mixed tolerance

[|O_{\mathrm{actual}}-O_{\mathrm{reference}}|\le 0.02+0.02|O_{\mathrm{reference}}|.]

Validation item	Evidence
Genuine cuTile smoke	PASS; exact check over 4,096 outputs
Actual-cuTile attention suite	48 passed, 0 failed
Legal case/tile accuracy configurations	29 passed, 0 tolerance violations
Maximum absolute output error	0.0009765625
Largest per-configuration P99 absolute error	0.00048828125
Largest per-configuration RMSE	0.000086860741
Maximum relative error where abs(reference) ≥ 0.01	0.02245863 (2.25%)
Maximum error / permitted tolerance	0.024366 (2.44% of tolerance budget)
CLI output	FP16, finite, shape-valid; NumPy oracle PASS

The 2.44% number is the fraction of the permitted tolerance envelope consumed by the worst element; it is not 2.44% output error. The CLI generated deterministic FP16 Q/K/V of shape [1,1,64,32], seed 20261018, and verified shape, dtype, finiteness and oracle agreement. Test and JSON/log evidence is preserved in core artifact #12264746195.

4. Performance methodology and results

Seven fixed workloads and up to five legal tile candidates were tested: 32×64, 64×64, 64×128, 128×64 and 128×128. Each of three independent sessions used ten warmups and 30 CUDA-event samples per configuration. Each timed sample averaged five launches, for 150 timed launches per case/tile/session. Tile order was rotated/reversed between sessions. The deliberately unfused FP32 CuPy baseline was measured before and after the sweep to expose timing drift.

Workload	Tile	Fused median (ms)	Fused P90 (ms)	Unfused baseline (ms)	Speedup
Short D=32	32×64	0.015770	0.018473	0.297270	19.10×
Short D=64	64×64	0.016282	0.018862	0.296243	18.33×
Short multi-head D=64	32×64	0.016074	0.018810	0.280320	16.96×
Medium D=64	64×64	0.017171	0.017833	0.269562	16.25×
Medium D=128	64×64	0.058368	0.058814	0.390861	6.70×
Long D=128	64×64	0.267059	0.269128	4.390656	16.49×
Long batched D=128	64×128	0.915558	0.921457	17.104077	18.69×

The general default remains 64×64. In the latest run, long D=128 / 64×64 (0.267059 ms) was about 1.42% faster than 64×128 (0.270848 ms). For longer-batched D=128, 64×128 was about 1.86% faster than 64×64 (0.915558 vs 0.932966 ms), with a 0.50% session range and 0.08% maximum baseline drift. Keep 64×128 as a workload-specific tile, not a universal default.

Short-case numbers are not reliable headline results: Windows/WDDM scheduling and launch overhead produce up to 197% fused range and 81.17% baseline drift. Output throughput in the benchmark JSON is based on output bytes; do not describe it as total memory bandwidth.

Cold/JIT timing caveat

The harness records first-call wall time separately when available, but these values may include compiler cache hits and are JIT-associated latency, not pure compile time. Medium D=128 first calls were roughly 27–32 ms. For long D=128 and long-batched D=128, the session-level JIT fields are null because those specializations were already compiled. A deliberately controlled clean-cache measurement, or an approved explanation for omitting it, remains open.

5. Nsight Compute profiling and interpretation

Nsight Compute CLI 2026.3.1.0 successfully captured four reports: basic throughput/resource profiles and supplemental SchedulerStats/WarpStateStats profiles for medium_d128, tile 64×64, and longer_batched_d128, tile 64×128. The profile artifact includes compressed .ncu-repz files, Details exports, Raw CSV files and status/command logs. The downloaded 289,372-byte ZIP matched GitHub's SHA-256 digest fd947285bc52731a97127fda08a86a9e53dc9a8000c9164966165fed0fc06b1c.

Metric	medium D=128, 64×64	long batched D=128, 64×128
Tensor shape	B=1,H=8,N=512,D=128	B=2,H=8,N=2048,D=128
NCU kernel duration	52.90 µs	984.26 µs
Grid / threads per block	64 CTAs / 128	512 CTAs / 128
Registers per thread	231	255
Static shared memory per CTA	65.54 KiB	81.92 KiB
Allocated shared memory per CTA	~66.56 KiB	~82.94 KiB
Theoretical / achieved occupancy	8.33% / 8.35%	8.33% / 8.33%
SM compute throughput	18.81% peak	30.27% peak
DRAM throughput	22.88% peak	11.89% peak
L1/TEX / L2 throughput	17.35% / 14.52%	21.00% / 20.07%
Scheduler cycles with no eligible warp	85.83%	85.26%
Eligible / active warps per scheduler	0.14 / 1.00	0.15 / 1.00
Warp cycles per issued instruction	7.08	6.78
NCU rule pipeline-stall estimate	2.4 cycles	3.1 cycles
NCU rule estimated local speedup	33.57% heuristic	46.04% heuristic

Interpretation: shared memory is the clearest CTA-residency constraint. The occupancy section attributes one CTA per SM and roughly 8.3% occupancy to the kernels' shared-memory footprint. Register usage is also high. Neither workload approaches peak compute or DRAM throughput, and only 14–15% of scheduler cycles have one or more eligible warps. Evidence therefore motivates hypothesis-driven experiments on shared-memory/register footprint and instruction scheduling.

The NCU "estimated local speedup" values are rule heuristics, not measured gains or guarantees. The optional smsp__pcsamp_sample_count metric was unavailable, so detailed PC/source-level stall attribution is not supported. The dedicated profiler artifact includes seven verified screenshots of the genuine Nsight Compute window, alongside native reports and metric exports. A matched medium hint-10 comparison has not completed, so that timing candidate remains unpromoted.

6. Optimization history and engineering decisions

Two optimizations were retained only after correctness checks passed:

1. **Causal-loop pruning:** skip K/V tiles wholly beyond the last causal position instead of loading them and masking all their scores.
2. **Mixed-precision P×V operands:** probabilities are rounded to FP16 operands for the second matrix multiply while QK accumulation, online-softmax state, normalizers and P×V accumulator remain FP32.

No global kernel default was changed solely from the hint timing. The medium workload's `LOAD_LATENCY=10` candidate improved median from 0.058342 ms to 0.053248 ms (8.73%) with 0.53% across-session ranges, after all 36 timing trials passed the NumPy accuracy gate. The matched NCU hint-10 report remains queued; inspect register/shared-memory/occupancy changes before recommending a workload-scoped setting. Week 3's profiler-guided improvement is therefore a measured candidate, not a closed milestone.

7. Assignment closure, limitations and next actions

Rubric item	Current status	Required ne
Fused causal cuTile kernel, online softmax, no global N×N matrix	PASS	Keep as bas
D=32/64/128, FP16 input/output, FP32 accumulation	PASS in tested matrix	Preserve va
Independent correctness and error report	PASS: 48 tests + 29 configs	Keep raw lo

Rubric item	Current status	Required ne
CLI and NumPy oracle	PASS	Keep deter
Unfused baseline, multiple tiles, 3 controlled sessions	PASS with caveats	Preserve JS
Two real cuTile workload profiles	PASS	Keep comp
Cold/JIT timings for each workload	PARTIAL	Clean-cach
Framework/vendor attention performance reference	BLOCKED before benchmark	Pinned PyT
Week 2 profiler-guided optimization	PARTIAL	Timing poi
Week 3 second profiler-guided improvement	CANDIDATE; NCU gate open	Review hint
Final report 5–8 pages	7-page PDF prepared	Confirm rec
Individual contributions	NEEDS OWNER INPUT	Insert actua

Remaining and optional objectives

The assignment's possible next step is to integrate the fused kernel into an example PyTorch model and measure output agreement and end-to-end improvement over that model's baseline. Other optional extensions identified in the project materials are BF16/FP8, backward propagation, variable-length/packed sequences, GQA/MQA, autotuning, CuTe DSL comparison (instructor-approved alternative track), and architecture-specific pipelining/async data movement. These should not displace the remaining rubric gaps. Do not install PyTorch or other packages without explicit approval.

Reproducibility links

- Latest native RTX 4060 validation run #39422402438
- Current core artifact #12264746195
- Dedicated Nsight Compute run #39414502490
- Dedicated NCU artifact #12261445684
- Detailed numerical/benchmark snapshot for 20 October
- Current experiment report and blockers
- Profiler analysis
- Rubric-mapped checklist
- Original problem description: nvidia2-cutile.pdf in repository root.

Individual contributions: [Owner/team to confirm. No authorship attribution is guessed in this report.]